Tests with Multi-Agent RAG Prototype

The file for the agent-RAG implementation (**lg_agent.py**) can be found here.

Bugs:

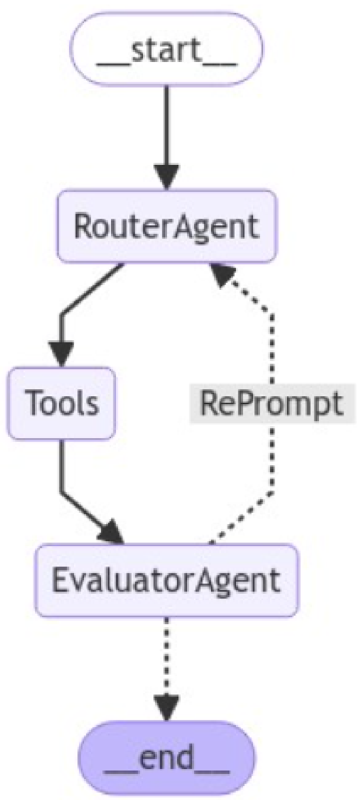
- Upon aggregation operations, if retrieved value was "0", the Agent assumes that the answer to the question has been determined, and does not re-prompt to try with the other tools, despite modifying the prompt.
  - E.g. How many ports does Router ... have? KG returned: 0 (as data was not present)
    - Expected outcome: The vector_search_process would be initiated, as there was insufficient context from the KG.
    - Actual outcome: "There are 0 ports for Router ...".

To work on:

- A for-loop based conditional would be possible, which would aim at running both tools to get maximum context, however it is not adaptive to the question asked, and the retrieval approach should ideally be identified and adjusted by the agent itself.
- State with defined attributes (.with_structured_output) can be transferred to have better control over amount of times each tool is called, for retrieving context.

Useful Resources:
langchain-ai/langgraph · GitHub
langgraph/examples/rag/langgraph_crag.ipynb at langchain-ai/langgraph (github.com)
knowledge-graph based RAG implementation - Neo4J

LangGraph Flow Diagram



Question	Logs from running Agent Graph
QUESTION >> What is the product number of Baseband 6303?	**-------------------TOOL CALLED >> kg_query_process ---------------------** **> Entering new KnowledgeGraphSparqlQAChain chain...** **Identified intent:** **SELECT** **> Entering new LLMChain chain...** **Prompt after formatting:** **Task: Generate a SPARQL SELECT statement for querying a graph database.** **Here is a few Examples you can use for reference, and adjust according to the user question:** **Example 2: To find the product number of any product ->**

Question: What is the product number of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?product_number
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
   ?prod rdfs:label ?product_label .
   FILTER(STR(?product_label) = "Baseband 6318")
   ?prod eva:has_product_number ?product_number .
   }
}
```

Example 3:
Question: How to find all the details of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?product ?book ?title ?section
  (GROUP_CONCAT(DISTINCT ?video_x; separator="|||") AS ?video)
  (GROUP_CONCAT(DISTINCT ?eplay_url_x; separator="|||") AS ?eplay_url)
WHERE {
   {
    SELECT DISTINCT ?prod
    WHERE {
    GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
     ?prod ^cpi:has_category/cpi:has_category ?prod .
     ?prod rdfs:label ?product_label .
     FILTER(STR(?product_label) = "Baseband 6318")
     }
    }
    LIMIT 1
   }
   ?prod skos:prefLabel ?product .
   ?prod ^cpi:has_category ?task .
   ?task a cpi:task ;
        ^cpi:contains ?bookmap .
   ?bookmap cpi:booktitle ?book .
   FILTER REGEX(?book,'install','i')
   ?task cpi:text ?title ;
        cpi:section ?sectionstr .
   BIND(xsd:float(?sectionstr) AS ?section)
}
GROUP BY ?product ?book ?title ?section
ORDER BY ?book ?section
```
Example 4:
Question: How to install Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX text: <http://jena.apache.org/text#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>
```

```
SELECT DISTINCT ?book ?title ?step_index ?step_text (GROUP_CONCAT(DISTINCT ?image; separator="|||") AS ?
images) ?note ?admonitions ?task_context
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
   ?prod rdfs:label ?product_label .
   FILTER(STR(?product_label) = "Baseband 6318")
   ?prod skos:prefLabel ?product .
   ?task cpi:has_category ?prod .
   ?task a cpi:task .
   ?task cpi:admonitions ?admonitions .
   ?task cpi:text ?title .
   ?task rdfs:label ?label .
   ?task cpi:contains ?step .
   ?step cpi:index ?step_index .
   ?step cpi:text ?step_text .
   ?bookmap cpi:contains ?task .
   ?bookmap cpi:booktitle ?book .
   OPTIONAL {
     ?step cpi:has_image ?img .
     ?img cpi:uri ?im .
   }

   OPTIONAL {
     ?step cpi:note ?note .
     ?task cpi:task_context ?task_context .
   }
   BIND(COALESCE(?im, "Has no image") AS ?image)

   FILTER (?book  = "Install Baseband")
   }
}
GROUP BY ?book ?title ?step_index ?step_text ?note ?admonitions ?task_context
ORDER BY ?book ?title ?step_index
LIMIT 30
```
Example 5:
Question: What optical indicators does Baseband 5212 have?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?prod ?text ?opticalIndicator WHERE {
   GRAPH <http://www.ericsson.com/eva/graph/hw-base> {
     # Two ways to retrieve a baseband
     # Approach 1: From textual fields
     ?prod eva:has_category eva:baseband_6318 .
     ?prod eva:contains ?instructions .
     ?instructions eva:contains ?steps .
     ?steps eva:text ?text .
     FILTER(REGEX(?text, "optical indicator", "i"))
     # Approach 2: Retrieve the blank nodes, which store optical indicator information (not all models have this)
     OPTIONAL {
        ?prod rdfs:label ?product_label .
        FILTER(CONTAINS(?product_label, "radio_access"))
        ?prod rdfs:subClassOf [ owl:onProperty eva:hasOpticalIndicator ; owl:someValuesFrom ?opticalIndicator ] .
     }
   }
}
```
Example 6:
Question: Does Baseband 6641 support NR?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>
```

```
SELECT ?rat
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
   ?prod rdfs:label ?product_label .
   FILTER(STR(?product_label) = "Baseband 6641")
   ?prod rdfs:subClassOf [ owl:onProperty eva:serves_rat ; owl:someValuesFrom ?rat_iri ] .
   ?rat_iri skos:prefLabel ?rat .
   }
}
```

Example 7:
Question: How to power up Baseband 6318?
The following query within backticks ``` and ``` would be the answer:

```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eric: <http://www.ericsson.com/eva#>

SELECT ?name ?index ?description ?subpage WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
  ?prod rdf:type eric:document.
  ?prod rdf:type eric:topic.
  ?prod eric:has_category eric:baseband_6318.
  ?prod eric:text ?text.
  FILTER(REGEX(?text, "[Pp]ower[ _][Uu]p") ).
  ?prod eric:contains ?subpage.
  ?prod eric:text ?name.
  ?subpage eric:index ?index.
  ?subpage eric:text ?description
  }
}
  ORDER BY ?name ?index
```

Instructions:
Use only the node types, object properties and data properties provided in the below schema.
Do not use any node types and properties that are not explicitly provided in below schema.
The object value after the predicate must not be a string, and should be filtered using the FILTER command.
All generated queries must look through all subgraphs, by using "GRAPH ?g".

The Schema is in JSON format.
Include only necessary prefixes based on the below schema.
Never use any undefined prefixes.

Schema:

The OWL graph use the following prefixes:
{'owl:': 'http://www.w3.org/2002/07/owl#', 'rdf:': 'http://www.w3.org/1999/02/22-rdf-syntax-ns#', 'rdfs:': 'http://www.w3.org/2000/01/rdf-schema#', 'xsd:': 'http://www.w3.org/2001/XMLSchema#', 'eva:': 'http://www.ericsson.com/eva#', 'skos:': 'http://www.w3.org/2004/02/skos/core#'}
The OWL graph supports the following node types:

{
 "uri": "owl:AnnotationProperty",
"comment": "None"
},
{
 "uri": "owl:Ontology",
"comment": "None"
},
{
 "uri": "eva:bookmap",
"comment": "None"
},
{
 "uri": "eva:document",
"comment": "None"
},
{
 "uri": "eva:concept",
"comment": "None"
},
{
 "uri": "eva:topic",
"comment": "None"
},
{

"uri": "eva:task",
"comment": "None"
},
{
"uri": "eva:fragment",
"comment": "None"
},
{
"uri": "eva:step",
"comment": "None"
},
{
"uri": "eva:product_name_product",
"comment": "None"
},
{
"uri": "owl:Class",
"comment": "None"
},
{
"uri": "eva:product_number_product",
"comment": "None"
},
{
"uri": "eva:product_number_rstate_product",
"comment": "None"
},
{
"uri": "owl:DatatypeProperty",
"comment": "None"
},
{
"uri": "rdf:Seq",
"comment": "None"
},
{
"uri": "owl:ObjectProperty",
"comment": "None"
},
{
"uri": "owl:TransitiveProperty",
"comment": "None"
},
{
"uri": "eva:image",
"comment": "None"
},
{
"uri": "eva:p3d",
"comment": "None"
},
{
"uri": "eva:tool",
"comment": "None"
},
{
"uri": "eva:video",
"comment": "None"
},
{
"uri": "owl:ontology",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#log",
"comment": "None"
},
{
"uri": "owl:Restriction",
"comment": "None"
}
The OWL graph supports the following object properties, i.e., relationships between objects:

{
"uri": "eva:contains",
"comment": "None"
},
{

"uri": "eva:hasColor",
"comment": "None"
},
{
"uri": "eva:hasFeaures",
"comment": "None"
},
{
"uri": "eva:hasIndicatorMode",
"comment": "None"
},
{
"uri": "eva:hasMarking",
"comment": "None"
},
{
"uri": "eva:hasOpticalIndicator",
"comment": "None"
},
{
"uri": "eva:hasPart",
"comment": "None"
},
{
"uri": "eva:hasPart_directly",
"comment": "None"
},
{
"uri": "eva:hasProductType",
"comment": "None"
},
{
"uri": "eva:hasSymbol",
"comment": "None"
},
{
"uri": "eva:has_alarm",
"comment": "None"
},
{
"uri": "eva:has_category",
"comment": "None"
},
{
"uri": "eva:has_commissioning_procedure",
"comment": "None"
},
{
"uri": "eva:has_component",
"comment": "New Product built from to other Products"
},
{
"uri": "eva:has_figure_id",
"comment": "None"
},
{
"uri": "eva:has_grounding_guideline",
"comment": "None"
},
{
"uri": "eva:has_image",
"comment": "None"
},
{
"uri": "eva:has_image_id",
"comment": "None"
},
{
"uri": "eva:has_managed_object",
"comment": "None"
},
{
"uri": "eva:has_mounting_alternative",
"comment": "Information on mounting posssibility (Wall or Pole, etc.)"
},
{
"uri": "eva:has_overview",

"comment": "None"
},
{
 "uri": "eva:has_product",
"comment": "None"
},
{
 "uri": "eva:has_product_type",
"comment": "Product Type Classification"
},
{
 "uri": "eva:has_video",
"comment": "None"
},
{
 "uri": "eva:hastoolList",
"comment": "None"
},
{
 "uri": "eva:is_contained_by",
"comment": "None"
},
{
 "uri": "eva:is_installed_as",
"comment": "None"
},
{
 "uri": "eva:is_installed_in",
"comment": "Environment (Indoor or Outdor)"
},
{
 "uri": "eva:is_raised_by",
"comment": "None"
},
{
 "uri": "eva:item_number",
"comment": "None"
},
{
 "uri": "eva:needTools",
"comment": "None"
},
{
 "uri": "eva:partOf",
"comment": "None"
},
{
 "uri": "eva:raises_alarm",
"comment": "None"
},
{
 "uri": "eva:refers_to",
"comment": "None"
},
{
 "uri": "eva:serves_rat",
"comment": "Radio Access Technology (RAT)"
},
{
 "uri": "eva:toolImage",
"comment": "None"
},
{
 "uri": "eva:toolName",
"comment": "None"
},
{
 "uri": "eva:usesOpticalIndicator",
"comment": "None"
},
{
 "uri": "http://www.ericsson.com/eva/log#includes_file",
"comment": "None"
}
The OWL graph supports the following data properties, i.e., relationships between objects and literals:

{
 "uri": "eva:additional_text",

"comment": "None"
},
{
 "uri": "eva:admonitions",
"comment": "None"
},
{
 "uri": "eva:book_pdf",
"comment": "None"
},
{
 "uri": "eva:bookdate",
"comment": "None"
},
{
 "uri": "eva:bookmeta",
"comment": "None"
},
{
 "uri": "eva:booknumber",
"comment": "None"
},
{
 "uri": "eva:booktitle",
"comment": "None"
},
{
 "uri": "eva:booktitlealt",
"comment": "None"
},
{
 "uri": "eva:has_image",
"comment": "None"
},
{
 "uri": "eva:has_product_number",
"comment": "FAJ Number of Resource"
},
{
 "uri": "eva:index",
"comment": "None"
},
{
 "uri": "eva:on_site_activity",
"comment": "None"
},
{
 "uri": "eva:overview_context",
"comment": "None"
},
{
 "uri": "eva:p3d_file",
"comment": "None"
},
{
 "uri": "eva:p3d_tag",
"comment": "None"
},
{
 "uri": "eva:p3d_url",
"comment": "None"
},
{
 "uri": "eva:placement",
"comment": "None"
},
{
 "uri": "eva:revision",
"comment": "None"
},
{
 "uri": "eva:section",
"comment": "None"
},
{
 "uri": "eva:task_context",
"comment": "None"

},
{
"uri": "eva:topic",
"comment": "None"
},
{
"uri": "eva:video_file",
"comment": "None"
},
{
"uri": "eva:video_tag",
"comment": "None"
},
{
"uri": "eva:video_url",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#file_sha-256",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#filename",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#source_version",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#time_elapsed",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#time_elapsed_(text)",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#timestamp",
"comment": "None"
}
The OWL graph supports the following annotation properties, i.e., comments or information about the nodes:

{
"uri": "http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled",
"comment": "None"
},
{
"uri": "eva:note",
"comment": "None"
},
{
"uri": "eva:text",
"comment": "None"
},
{
"uri": "eva:uri",
"comment": "None"
},
{
"uri": "skos:prefLabel",
"comment": "None"
}

Note: Be as concise as possible.
Do not include any explanations or assumptions or suggestions or notes or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except the SPARQL query generated within backticks ``` and ```.
Do not include the backticks in the response. Only return the SPARQL query within backticks.

History:
[]

The question is:
What is the product number of Baseband 6303?

	> Finished chain. Generated SPARQL: PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> PREFIX eva: <http://www.ericsson.com/eva#> SELECT ?product_number WHERE { GRAPH <http://www.ericsson.com/eva/graph/hw-base>{ ?prod rdfs:label ?product_label . FILTER(STR(?product_label) = "Baseband 6303") ?prod eva:has_product_number ?product_number . } } Full Context: [{'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}, {'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}, {'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}, {'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}, {'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}, {'product_number': {'type': 'literal', 'value': 'KDU137974/11'}}] > Finished chain. ------------------------ Question: What is the product number of Baseband 6303? Last Answer: The product number of Baseband 6303 is 'KDU137974/11'. Last tool used: kg_query_process Whether the question is answered: ANSWERED ------------------------
QUESTION >> How many r-states does MINI-LINK 622222222 have?	------------------TOOL CALLED >> retrieve_process -------------------- ------------------------ Question: How many r-states does MINI-LINK 622222222 have? Last Answer: I'm sorry, but I don't have any specific information about the number of r-states for the MINI-LINK 622222222. Please refer to the product's technical documentation or contact the manufacturer for accurate information. Last tool used: retrieve_process Whether the question is answered: NONE ------------------------ ------------------TOOL CALLED >> kg_query_process -------------------- > Entering new KnowledgeGraphSparqlQAChain chain... Identified intent: SELECT > Entering new LLMChain chain... Prompt after formatting: Task: Generate a SPARQL SELECT statement for querying a graph database. Here is a few Examples you can use for reference, and adjust according to the user question: Example 2: To find the product number of any product -> Question: What is the product number of Baseband 6318? The following query within backticks ``` and ``` would be the answer: ``` PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#> PREFIX eva: <http://www.ericsson.com/eva#> SELECT ?product_number WHERE { GRAPH <http://www.ericsson.com/eva/graph/hw-base>{ ?prod rdfs:label ?product_label . FILTER(STR(?product_label) = "Baseband 6318") ?prod eva:has_product_number ?product_number . } } ```

Example 3:
Question: How to find all the details of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?product ?book ?title ?section
 (GROUP_CONCAT(DISTINCT ?video_x; separator="|||") AS ?video)
 (GROUP_CONCAT(DISTINCT ?eplay_url_x; separator="|||") AS ?eplay_url)
WHERE {
  {
   SELECT DISTINCT ?prod
   WHERE {
   GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod ^cpi:has_category/cpi:has_category ?prod .
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    }
   }
   LIMIT 1
  }
  ?prod skos:prefLabel ?product .
  ?prod ^cpi:has_category ?task .
  ?task a cpi:task ;
      ^cpi:contains ?bookmap .
  ?bookmap cpi:booktitle ?book .
  FILTER REGEX(?book,'install','i')
  ?task cpi:text ?title ;
      cpi:section ?sectionstr .
  BIND(xsd:float(?sectionstr) AS ?section)
}
GROUP BY ?product ?book ?title ?section
ORDER BY ?book ?section
```
Example 4:
Question: How to install Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX text: <http://jena.apache.org/text#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?book ?title ?step_index ?step_text (GROUP_CONCAT(DISTINCT ?image; separator="|||") AS ?
images) ?note ?admonitions ?task_context
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
  ?prod rdfs:label ?product_label .
  FILTER(STR(?product_label) = "Baseband 6318")
  ?prod skos:prefLabel ?product .
  ?task cpi:has_category ?prod .
  ?task a cpi:task .
  ?task cpi:admonitions ?admonitions .
  ?task cpi:text ?title .
  ?task rdfs:label ?label .
  ?task cpi:contains ?step .
  ?step cpi:index ?step_index .
  ?step cpi:text ?step_text .
  ?bookmap cpi:contains ?task .
  ?bookmap cpi:booktitle ?book .
  OPTIONAL {
   ?step cpi:has_image ?img .
   ?img cpi:uri ?im .
  }
```

````
   OPTIONAL {
     ?step cpi:note ?note .
     ?task cpi:task_context ?task_context .
   }
   BIND(COALESCE(?im, "Has no image") AS ?image)

   FILTER (?book  = "Install Baseband")
   }
}
GROUP BY ?book ?title ?step_index ?step_text ?note ?admonitions ?task_context
ORDER BY ?book ?title ?step_index
LIMIT 30
```
Example 5:
Question: What optical indicators does Baseband 5212 have?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?prod ?text ?opticalIndicator WHERE {
   GRAPH <http://www.ericsson.com/eva/graph/hw-base> {
     # Two ways to retrieve a baseband
     # Approach 1: From textual fields
     ?prod eva:has_category eva:baseband_6318 .
     ?prod eva:contains ?instructions .
     ?instructions eva:contains ?steps .
     ?steps eva:text ?text .
     FILTER(REGEX(?text, "optical indicator", "i"))
     # Approach 2: Retrieve the blank nodes, which store optical indicator information (not all models have this)
     OPTIONAL {
        ?prod rdfs:label ?product_label .
        FILTER(CONTAINS(?product_label, "radio_access"))
        ?prod rdfs:subClassOf [ owl:onProperty eva:hasOpticalIndicator ; owl:someValuesFrom ?opticalIndicator ] .
     }
   }
}
```
Example 6:
Question: Does Baseband 6641 support NR?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?rat
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
   ?prod rdfs:label ?product_label .
   FILTER(STR(?product_label) = "Baseband 6641")
   ?prod rdfs:subClassOf [ owl:onProperty eva:serves_rat ; owl:someValuesFrom ?rat_iri ] .
   ?rat_iri skos:prefLabel ?rat .
   }
}
```
Example 7:
Question: How to power up Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eric: <http://www.ericsson.com/eva#>
````

SELECT ?name ?index ?description ?subpage WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
 ?prod rdf:type eric:document.
 ?prod rdf:type eric:topic.
 ?prod eric:has_category eric:baseband_6318.
 ?prod eric:text ?text.
 FILTER(REGEX(?text, "[Pp]ower[_][Uu]p")).
 ?prod eric:contains ?subpage.
 ?prod eric:text ?name.
 ?subpage eric:index ?index.
 ?subpage eric:text ?description
 }
}
 ORDER BY ?name ?index
```

Instructions:
Use only the node types, object properties and data properties provided in the below schema.
Do not use any node types and properties that are not explicitly provided in below schema.
The object value after the predicate must not be a string, and should be filtered using the FILTER command.
All generated queries must look through all subgraphs, by using "GRAPH ?g".

The Schema is in JSON format.
Include only necessary prefixes based on the below schema.
Never use any undefined prefixes.

Schema:

The OWL graph use the following prefixes:
{'owl:': 'http://www.w3.org/2002/07/owl#', 'rdf:': 'http://www.w3.org/1999/02/22-rdf-syntax-ns#', 'rdfs:': 'http://www.w3.org/2000/01/rdf-schema#', 'xsd:': 'http://www.w3.org/2001/XMLSchema#', 'eva:': 'http://www.ericsson.com/eva#', 'skos:': 'http://www.w3.org/2004/02/skos/core#'}
The OWL graph supports the following node types:

{
 "uri": "owl:AnnotationProperty",
"comment": "None"
},
{
 "uri": "owl:Ontology",
"comment": "None"
},
{
 "uri": "eva:bookmap",
"comment": "None"
},
{
 "uri": "eva:document",
"comment": "None"
},
{
 "uri": "eva:concept",
"comment": "None"
},
{
 "uri": "eva:topic",
"comment": "None"
},
{
 "uri": "eva:task",
"comment": "None"
},
{
 "uri": "eva:fragment",
"comment": "None"
},
{
 "uri": "eva:step",
"comment": "None"
},
{
 "uri": "eva:product_name_product",
"comment": "None"
},
{
 "uri": "owl:Class",
"comment": "None"
```

},
{
"uri": "eva:product_number_product",
"comment": "None"
},
{
"uri": "eva:product_number_rstate_product",
"comment": "None"
},
{
"uri": "owl:DatatypeProperty",
"comment": "None"
},
{
"uri": "rdf:Seq",
"comment": "None"
},
{
"uri": "owl:ObjectProperty",
"comment": "None"
},
{
"uri": "owl:TransitiveProperty",
"comment": "None"
},
{
"uri": "eva:image",
"comment": "None"
},
{
"uri": "eva:p3d",
"comment": "None"
},
{
"uri": "eva:tool",
"comment": "None"
},
{
"uri": "eva:video",
"comment": "None"
},
{
"uri": "owl:ontology",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#log",
"comment": "None"
},
{
"uri": "owl:Restriction",
"comment": "None"
}
The OWL graph supports the following object properties, i.e., relationships between objects:

{
"uri": "eva:contains",
"comment": "None"
},
{
"uri": "eva:hasColor",
"comment": "None"
},
{
"uri": "eva:hasFeaures",
"comment": "None"
},
{
"uri": "eva:hasIndicatorMode",
"comment": "None"
},
{
"uri": "eva:hasMarking",
"comment": "None"
},
{
"uri": "eva:hasOpticalIndicator",
"comment": "None"

},
{
"uri": "eva:hasPart",
"comment": "None"
},
{
"uri": "eva:hasPart_directly",
"comment": "None"
},
{
"uri": "eva:hasProductType",
"comment": "None"
},
{
"uri": "eva:hasSymbol",
"comment": "None"
},
{
"uri": "eva:has_alarm",
"comment": "None"
},
{
"uri": "eva:has_category",
"comment": "None"
},
{
"uri": "eva:has_commissioning_procedure",
"comment": "None"
},
{
"uri": "eva:has_component",
"comment": "New Product built from to other Products"
},
{
"uri": "eva:has_figure_id",
"comment": "None"
},
{
"uri": "eva:has_grounding_guideline",
"comment": "None"
},
{
"uri": "eva:has_image",
"comment": "None"
},
{
"uri": "eva:has_image_id",
"comment": "None"
},
{
"uri": "eva:has_managed_object",
"comment": "None"
},
{
"uri": "eva:has_mounting_alternative",
"comment": "Information on mounting posssibility (Wall or Pole, etc.)"
},
{
"uri": "eva:has_overview",
"comment": "None"
},
{
"uri": "eva:has_product",
"comment": "None"
},
{
"uri": "eva:has_product_type",
"comment": "Product Type Classification"
},
{
"uri": "eva:has_video",
"comment": "None"
},
{
"uri": "eva:hastoolList",
"comment": "None"
},

{
"uri": "eva:is_contained_by",
"comment": "None"
},
{
"uri": "eva:is_installed_as",
"comment": "None"
},
{
"uri": "eva:is_installed_in",
"comment": "Environment (Indoor or Outdor)"
},
{
"uri": "eva:is_raised_by",
"comment": "None"
},
{
"uri": "eva:item_number",
"comment": "None"
},
{
"uri": "eva:needTools",
"comment": "None"
},
{
"uri": "eva:partOf",
"comment": "None"
},
{
"uri": "eva:raises_alarm",
"comment": "None"
},
{
"uri": "eva:refers_to",
"comment": "None"
},
{
"uri": "eva:serves_rat",
"comment": "Radio Access Technology (RAT)"
},
{
"uri": "eva:toolImage",
"comment": "None"
},
{
"uri": "eva:toolName",
"comment": "None"
},
{
"uri": "eva:usesOpticalIndicator",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#includes_file",
"comment": "None"
}
The OWL graph supports the following data properties, i.e., relationships between objects and literals:

{
"uri": "eva:additional_text",
"comment": "None"
},
{
"uri": "eva:admonitions",
"comment": "None"
},
{
"uri": "eva:book_pdf",
"comment": "None"
},
{
"uri": "eva:bookdate",
"comment": "None"
},
{
"uri": "eva:bookmeta",
"comment": "None"
},

{
"uri": "eva:booknumber",
"comment": "None"
},
{
"uri": "eva:booktitle",
"comment": "None"
},
{
"uri": "eva:booktitlealt",
"comment": "None"
},
{
"uri": "eva:has_image",
"comment": "None"
},
{
"uri": "eva:has_product_number",
"comment": "FAJ Number of Resource"
},
{
"uri": "eva:index",
"comment": "None"
},
{
"uri": "eva:on_site_activity",
"comment": "None"
},
{
"uri": "eva:overview_context",
"comment": "None"
},
{
"uri": "eva:p3d_file",
"comment": "None"
},
{
"uri": "eva:p3d_tag",
"comment": "None"
},
{
"uri": "eva:p3d_url",
"comment": "None"
},
{
"uri": "eva:placement",
"comment": "None"
},
{
"uri": "eva:revision",
"comment": "None"
},
{
"uri": "eva:section",
"comment": "None"
},
{
"uri": "eva:task_context",
"comment": "None"
},
{
"uri": "eva:topic",
"comment": "None"
},
{
"uri": "eva:video_file",
"comment": "None"
},
{
"uri": "eva:video_tag",
"comment": "None"
},
{
"uri": "eva:video_url",
"comment": "None"
},
{

"uri": "http://www.ericsson.com/eva/log#file_sha-256",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#filename",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#source_version",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#time_elapsed",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#time_elapsed_(text)",
"comment": "None"
},
{
"uri": "http://www.ericsson.com/eva/log#timestamp",
"comment": "None"
}
The OWL graph supports the following annotation properties, i.e., comments or information about the nodes:

{
"uri": "http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled",
"comment": "None"
},
{
"uri": "eva:note",
"comment": "None"
},
{
"uri": "eva:text",
"comment": "None"
},
{
"uri": "eva:uri",
"comment": "None"
},
{
"uri": "skos:prefLabel",
"comment": "None"
}

Note: Be as concise as possible.
Do not include any explanations or assumptions or suggestions or notes or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except the SPARQL query generated within backticks ``` and ```.
Do not include the backticks in the response. Only return the SPARQL query within backticks.

History:
[HumanMessage(content='What is the product number of Baseband 6303?'), AIMessage(content="The product number of Baseband 6303 is 'KDU137974/11'.")]

The question is:
How many r-states does MINI-LINK 622222222 have?

> Finished chain.
Generated SPARQL:

```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT (COUNT(?rstate) AS ?rstate_count)
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
   ?prod rdfs:label ?product_label .
   FILTER(STR(?product_label) = "MINI-LINK 622222222")
   ?prod eva:has_rstate ?rstate .
   }
}
```

Full Context:
[{'rstate_count': {'type': 'literal', 'datatype': 'http://www.w3.org/2001/XMLSchema#integer', 'value': '0'}}]

	> Finished chain. **------------------------** **Question: How many r-states does MINI-LINK 622222222 have?** **Last Answer: The MINI-LINK 622222222 has 0 r-states.** **Last tool used (from [retrieve_process, kg_query_process], if both tools have been executed for this question, add 'both' here): kg_query_process** **Whether the question is answered (then add "answered" in UPPERCASE here), or you don't know the answer, or there is no answer (then add "none" in UPPERCASE here): ANSWERED** **------------------------**

More questions:

What is the weight of AIR 21 B4A B12P B5P?

```
The question is:
What is the weight of AIR 21 B4A B12P B5P?

-------------------TOOL CALLED >> kg_query_process ---------------------

> Entering new KnowledgeGraphSparqlQAChain chain...

Identified intent:
SELECT

> Entering new LLMChain chain...
...
> Finished chain.
Generated SPARQL:
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?weight
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "AIR 21 B4A B12P B5P")
    ?prod eva:has_weight ?weight .
    }
}
Full Context:
[]

> Finished chain.
-------------------------
Question: 'What is the weight of AIR 21 B4A B12P B5P?'
Last Answer: "I'm sorry, but I don't have any information regarding the weight of AIR 21 B4A B12P B5P."
Last tool used: 'kg_query_process'
Whether the question is answered (then add "answered" in UPPERCASE here), or you don't know the answer, or
there is no answer (then add "none" in UPPERCASE here): NONE
-------------------------

-------------------TOOL CALLED >> vector_search_process ---------------------

-------------------------
Question: 'What is the weight of AIR 21 B4A B12P B5P?'
Last Answer: 'The weight of the AIR 21 B4A B12P B5P is 57000 grams.'
Last tool used: 'vector_search_process'
Whether the question is answered (then add "answered" in UPPERCASE here), or you don't know the answer, or
there is no answer (then add "none" in UPPERCASE here): ANSWERED
-------------------------
```

What is its maximum frequency?

```
QUESTION >> What is its maximum frequency?

-------------------TOOL CALLED >> kg_query_process ---------------------
```

```
> Entering new KnowledgeGraphSparqlQAChain chain...

Identified intent:
SELECT


> Entering new LLMChain chain...
Prompt after formatting:
Task: Generate a SPARQL SELECT statement for querying a graph database.
Here is a few Examples you can use for reference, and adjust according to the user question:

Example 2: To find the product number of any product ->

Question: What is the product number of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?product_number
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    ?prod eva:has_product_number ?product_number .
    }
}
```

Example 3:
Question: How to find all the details of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?product ?book ?title ?section
  (GROUP_CONCAT(DISTINCT ?video_x; separator="|||") AS ?video)
  (GROUP_CONCAT(DISTINCT ?eplay_url_x; separator="|||") AS ?eplay_url)
WHERE {
    {
      SELECT DISTINCT ?prod
      WHERE {
      GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
        ?prod ^cpi:has_category/cpi:has_category ?prod .
        ?prod rdfs:label ?product_label .
        FILTER(STR(?product_label) = "Baseband 6318")
        }
      }
      LIMIT 1
    }
    ?prod skos:prefLabel ?product .
    ?prod ^cpi:has_category ?task .
    ?task a cpi:task ;
          ^cpi:contains ?bookmap .
    ?bookmap cpi:booktitle ?book .
    FILTER REGEX(?book,'install','i')
    ?task cpi:text ?title ;
          cpi:section ?sectionstr .
    BIND(xsd:float(?sectionstr) AS ?section)
}
GROUP BY ?product ?book ?title ?section
ORDER BY ?book ?section
```
Example 4:
Question: How to install Baseband 6318?
```

````
The following query within backticks ``` and ``` would be the answer:
```
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX text: <http://jena.apache.org/text#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?book ?title ?step_index ?step_text (GROUP_CONCAT(DISTINCT ?image; separator="|||") AS ?images)
?note ?admonitions ?task_context
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    ?prod skos:prefLabel ?product .
    ?task cpi:has_category ?prod .
    ?task a cpi:task .
    ?task cpi:admonitions ?admonitions .
    ?task cpi:text ?title .
    ?task rdfs:label ?label .
    ?task cpi:contains ?step .
    ?step cpi:index ?step_index .
    ?step cpi:text ?step_text .
    ?bookmap cpi:contains ?task .
    ?bookmap cpi:booktitle ?book .
    OPTIONAL {
      ?step cpi:has_image ?img .
      ?img cpi:uri ?im .
    }

    OPTIONAL {
      ?step cpi:note ?note .
      ?task cpi:task_context ?task_context .
    }
    BIND(COALESCE(?im, "Has no image") AS ?image)

    FILTER (?book  = "Install Baseband")
    }
}
GROUP BY ?book ?title ?step_index ?step_text ?note ?admonitions ?task_context
ORDER BY ?book ?title ?step_index
LIMIT 30
```
Example 5:
Question: What optical indicators does Baseband 5212 have?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?prod ?text ?opticalIndicator WHERE {
    GRAPH <http://www.ericsson.com/eva/graph/hw-base> {
      # Two ways to retrieve a baseband
      # Approach 1: From textual fields
      ?prod eva:has_category eva:baseband_6318 .
      ?prod eva:contains ?instructions .
      ?instructions eva:contains ?steps .
      ?steps eva:text ?text .
      FILTER(REGEX(?text, "optical indicator", "i"))
      # Approach 2: Retrieve the blank nodes, which store optical indicator information (not all models have
this)
      OPTIONAL {
          ?prod rdfs:label ?product_label .
          FILTER(CONTAINS(?product_label, "radio_access"))
          ?prod rdfs:subClassOf [ owl:onProperty eva:hasOpticalIndicator ; owl:someValuesFrom ?opticalIndicator
] .
      }
````
````

````
    }
}
```
Example 6:
Question: Does Baseband 6641 support NR?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?rat
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6641")
    ?prod rdfs:subClassOf [ owl:onProperty eva:serves_rat ; owl:someValuesFrom ?rat_iri ] .
    ?rat_iri skos:prefLabel ?rat .
    }
}
```
Example 7:
Question: How to power up Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eric: <http://www.ericsson.com/eva#>

SELECT ?name ?index ?description ?subpage WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
  ?prod rdf:type eric:document.
  ?prod rdf:type eric:topic.
  ?prod eric:has_category eric:baseband_6318.
  ?prod eric:text ?text.
  FILTER(REGEX(?text, "[Pp]ower[ _][Uu]p") ).
  ?prod eric:contains ?subpage.
  ?prod eric:text ?name.
  ?subpage eric:index ?index.
  ?subpage eric:text ?description
  }
}
  ORDER BY ?name ?index
```

Instructions:
Use only the node types, object properties and data properties provided in the below schema.
Do not use any node types and properties that are not explicitly provided in below schema.
The object value after the predicate must not be a string, and should be filtered using the FILTER command.
All generated queries must look through all subgraphs, by using "GRAPH ?g".

The Schema is in JSON format.
Include only necessary prefixes based on the below schema.
Never use any undefined prefixes.

Schema:
The OWL graph use the following prefixes:
{'owl:': 'http://www.w3.org/2002/07/owl#', 'rdf:': 'http://www.w3.org/1999/02/22-rdf-syntax-ns#', 'rdfs:':
'http://www.w3.org/2000/01/rdf-schema#', 'xsd:': 'http://www.w3.org/2001/XMLSchema#', 'eva:': 'http://www.
ericsson.com/eva#', 'skos:': 'http://www.w3.org/2004/02/skos/core#'}

The OWL graph supports the following node types:

{ "uri": "owl:AnnotationProperty", "comment": "None"},
{ "uri": "owl:Ontology", "comment": "None"},
{ "uri": "eva:bookmap", "comment": "None"},
{ "uri": "eva:document", "comment": "None"},
{ "uri": "eva:concept", "comment": "None"},
{ "uri": "eva:topic", "comment": "None"},
{ "uri": "eva:task", "comment": "None"},
{ "uri": "eva:fragment", "comment": "None"},
{ "uri": "eva:step", "comment": "None"},
````
````

{ "uri": "eva:product_name_product", "comment": "None"},
{ "uri": "owl:Class", "comment": "None"},
{ "uri": "eva:product_number_product", "comment": "None"},
{ "uri": "eva:product_number_rstate_product", "comment": "None"},
{ "uri": "owl:DatatypeProperty", "comment": "None"},
{ "uri": "rdf:Seq", "comment": "None"},
{ "uri": "owl:ObjectProperty", "comment": "None"},
{ "uri": "owl:TransitiveProperty", "comment": "None"},
{ "uri": "eva:image", "comment": "None"},
{ "uri": "eva:p3d", "comment": "None"},
{ "uri": "eva:tool", "comment": "None"},
{ "uri": "eva:video", "comment": "None"},
{ "uri": "owl:ontology", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#log", "comment": "None"},
{ "uri": "owl:Restriction", "comment": "None"}

The OWL graph supports the following object properties, i.e., relationships between objects:

{ "uri": "eva:contains", "comment": "None"},
{ "uri": "eva:hasColor", "comment": "None"},
{ "uri": "eva:hasFeaures", "comment": "None"},
{ "uri": "eva:hasIndicatorMode", "comment": "None"},
{ "uri": "eva:hasMarking", "comment": "None"},
{ "uri": "eva:hasOpticalIndicator", "comment": "None"},
{ "uri": "eva:hasPart", "comment": "None"},
{ "uri": "eva:hasPart_directly", "comment": "None"},
{ "uri": "eva:hasProductType", "comment": "None"},
{ "uri": "eva:hasSymbol", "comment": "None"},
{ "uri": "eva:has_alarm", "comment": "None"},
{ "uri": "eva:has_category", "comment": "None"},
{ "uri": "eva:has_commissioning_procedure", "comment": "None"},
{ "uri": "eva:has_component", "comment": "New Product built from to other Products"},
{ "uri": "eva:has_figure_id", "comment": "None"},
{ "uri": "eva:has_grounding_guideline", "comment": "None"},
{ "uri": "eva:has_image", "comment": "None"},
{ "uri": "eva:has_image_id", "comment": "None"},
{ "uri": "eva:has_managed_object", "comment": "None"},
{ "uri": "eva:has_mounting_alternative", "comment": "Information on mounting posssibility (Wall or Pole, etc.)"},
{ "uri": "eva:has_overview", "comment": "None"},
{ "uri": "eva:has_product", "comment": "None"},
{ "uri": "eva:has_product_type", "comment": "Product Type Classification"},
{ "uri": "eva:has_video", "comment": "None"},
{ "uri": "eva:hastoolList", "comment": "None"},
{ "uri": "eva:is_contained_by", "comment": "None"},
{ "uri": "eva:is_installed_as", "comment": "None"},
{ "uri": "eva:is_installed_in", "comment": "Environment (Indoor or Outdor)"},
{ "uri": "eva:is_raised_by", "comment": "None"},
{ "uri": "eva:item_number", "comment": "None"},
{ "uri": "eva:needTools", "comment": "None"},
{ "uri": "eva:partOf", "comment": "None"},
{ "uri": "eva:raises_alarm", "comment": "None"},
{ "uri": "eva:refers_to", "comment": "None"},
{ "uri": "eva:serves_rat", "comment": "Radio Access Technology (RAT)"},
{ "uri": "eva:toolImage", "comment": "None"},
{ "uri": "eva:toolName", "comment": "None"},
{ "uri": "eva:usesOpticalIndicator", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#includes_file", "comment": "None"}

The OWL graph supports the following data properties, i.e., relationships between objects and literals:

{ "uri": "eva:additional_text", "comment": "None"},
{ "uri": "eva:admonitions", "comment": "None"},
{ "uri": "eva:book_pdf", "comment": "None"},
{ "uri": "eva:bookdate", "comment": "None"},
{ "uri": "eva:bookmeta", "comment": "None"},
{ "uri": "eva:booknumber", "comment": "None"},
{ "uri": "eva:booktitle", "comment": "None"},
{ "uri": "eva:booktitlealt", "comment": "None"},
{ "uri": "eva:has_image", "comment": "None"},
{ "uri": "eva:has_product_number", "comment": "FAJ Number of Resource"},

```
{ "uri": "eva:index", "comment": "None"},
{ "uri": "eva:on_site_activity", "comment": "None"},
{ "uri": "eva:overview_context", "comment": "None"},
{ "uri": "eva:p3d_file", "comment": "None"},
{ "uri": "eva:p3d_tag", "comment": "None"},
{ "uri": "eva:p3d_url", "comment": "None"},
{ "uri": "eva:placement", "comment": "None"},
{ "uri": "eva:revision", "comment": "None"},
{ "uri": "eva:section", "comment": "None"},
{ "uri": "eva:task_context", "comment": "None"},
{ "uri": "eva:topic", "comment": "None"},
{ "uri": "eva:video_file", "comment": "None"},
{ "uri": "eva:video_tag", "comment": "None"},
{ "uri": "eva:video_url", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#file_sha-256", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#filename", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#source_version", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#time_elapsed", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#time_elapsed_(text)", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#timestamp", "comment": "None"}

The OWL graph supports the following annotation properties, i.e., comments or information about the nodes:

{ "uri": "http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled", "comment": "None"},
{ "uri": "eva:note", "comment": "None"},
{ "uri": "eva:text", "comment": "None"},
{ "uri": "eva:uri", "comment": "None"},
{ "uri": "skos:prefLabel", "comment": "None"}

Note: Be as concise as possible.
Do not include any explanations or assumptions or suggestions or notes or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except the SPARQL query generated within backticks ``` and ```.
Do not include the backticks in the response. Only return the SPARQL query within backticks.

History:
[HumanMessage(content='How many ports are in Router 6471/2 DC?'), AIMessage(content='The Router 6471/2 DC does
not have any ports.'), HumanMessage(content='What is the weight of AIR 21 B4A B12P B5P?'), AIMessage(content="
I'm sorry, but I don't have any information regarding the weight of AIR 21 B4A B12P B5P.")]

The question is:
What is its maximum frequency?


> Finished chain.
Generated SPARQL:
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?max_frequency
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    ?prod eva:has_max_frequency ?max_frequency .
    }
}
Full Context:
[]

> Finished chain.
-------------------------
Question: 'What is its maximum frequency?'
Last Answer: "I'm sorry, but I don't have any information regarding the maximum frequency."
Last tool used: 'kg_query_process'
Whether the question is answered (then add "answered" in UPPERCASE here), or you don't know the answer, or
there is no answer (then add "none" in UPPERCASE here): NONE
-------------------------

-------------------TOOL CALLED >> kg_query_process ---------------------
```

````
> Entering new KnowledgeGraphSparqlQAChain chain...
Identified intent:
SELECT


> Entering new LLMChain chain...
Prompt after formatting:
Task: Generate a SPARQL SELECT statement for querying a graph database.
Here is a few Examples you can use for reference, and adjust according to the user question:

Example 2: To find the product number of any product ->

Question: What is the product number of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?product_number
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    ?prod eva:has_product_number ?product_number .
    }
}
```

Example 3:
Question: How to find all the details of Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?product ?book ?title ?section
  (GROUP_CONCAT(DISTINCT ?video_x; separator="|||") AS ?video)
  (GROUP_CONCAT(DISTINCT ?eplay_url_x; separator="|||") AS ?eplay_url)
WHERE {
    {
      SELECT DISTINCT ?prod
      WHERE {
      GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
        ?prod ^cpi:has_category/cpi:has_category ?prod .
        ?prod rdfs:label ?product_label .
        FILTER(STR(?product_label) = "Baseband 6318")
        }
      }
      LIMIT 1
    }
    ?prod skos:prefLabel ?product .
    ?prod ^cpi:has_category ?task .
    ?task a cpi:task ;
          ^cpi:contains ?bookmap .
    ?bookmap cpi:booktitle ?book .
    FILTER REGEX(?book,'install','i')
    ?task cpi:text ?title ;
          cpi:section ?sectionstr .
    BIND(xsd:float(?sectionstr) AS ?section)
}
GROUP BY ?product ?book ?title ?section
ORDER BY ?book ?section
```
Example 4:
Question: How to install Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
````

````
```
PREFIX cpi: <http://www.ericsson.com/eva#>
PREFIX eg: <http://www.ericsson.com/eva/graph/>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX skos: <http://www.w3.org/2004/02/skos/core#>
PREFIX text: <http://jena.apache.org/text#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

SELECT DISTINCT ?book ?title ?step_index ?step_text (GROUP_CONCAT(DISTINCT ?image; separator="|||") AS ?images)
?note ?admonitions ?task_context
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6318")
    ?prod skos:prefLabel ?product .
    ?task cpi:has_category ?prod .
    ?task a cpi:task .
    ?task cpi:admonitions ?admonitions .
    ?task cpi:text ?title .
    ?task rdfs:label ?label .
    ?task cpi:contains ?step .
    ?step cpi:index ?step_index .
    ?step cpi:text ?step_text .
    ?bookmap cpi:contains ?task .
    ?bookmap cpi:booktitle ?book .
    OPTIONAL {
      ?step cpi:has_image ?img .
      ?img cpi:uri ?im .
    }

    OPTIONAL {
      ?step cpi:note ?note .
      ?task cpi:task_context ?task_context .
    }
    BIND(COALESCE(?im, "Has no image") AS ?image)

    FILTER (?book  = "Install Baseband")
    }
}
GROUP BY ?book ?title ?step_index ?step_text ?note ?admonitions ?task_context
ORDER BY ?book ?title ?step_index
LIMIT 30
```
Example 5:
Question: What optical indicators does Baseband 5212 have?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>

SELECT ?prod ?text ?opticalIndicator WHERE {
    GRAPH <http://www.ericsson.com/eva/graph/hw-base> {
      # Two ways to retrieve a baseband
      # Approach 1: From textual fields
      ?prod eva:has_category eva:baseband_6318 .
      ?prod eva:contains ?instructions .
      ?instructions eva:contains ?steps .
      ?steps eva:text ?text .
      FILTER(REGEX(?text, "optical indicator", "i"))
      # Approach 2: Retrieve the blank nodes, which store optical indicator information (not all models have
this)
      OPTIONAL {
          ?prod rdfs:label ?product_label .
          FILTER(CONTAINS(?product_label, "radio_access"))
          ?prod rdfs:subClassOf [ owl:onProperty eva:hasOpticalIndicator ; owl:someValuesFrom ?opticalIndicator
] .
      }
    }
````
````

````
}
```
Example 6:
Question: Does Baseband 6641 support NR?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?rat
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "Baseband 6641")
    ?prod rdfs:subClassOf [ owl:onProperty eva:serves_rat ; owl:someValuesFrom ?rat_iri ] .
    ?rat_iri skos:prefLabel ?rat .
    }
}
```
Example 7:
Question: How to power up Baseband 6318?
The following query within backticks ``` and ``` would be the answer:
```
PREFIX rdf: <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eric: <http://www.ericsson.com/eva#>

SELECT ?name ?index ?description ?subpage WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
  ?prod rdf:type eric:document.
  ?prod rdf:type eric:topic.
  ?prod eric:has_category eric:baseband_6318.
  ?prod eric:text ?text.
  FILTER(REGEX(?text, "[Pp]ower[ _][Uu]p") ).
  ?prod eric:contains ?subpage.
  ?prod eric:text ?name.
  ?subpage eric:index ?index.
  ?subpage eric:text ?description
  }
}
  ORDER BY ?name ?index
```

Instructions:
Use only the node types, object properties and data properties provided in the below schema.
Do not use any node types and properties that are not explicitly provided in below schema.
The object value after the predicate must not be a string, and should be filtered using the FILTER command.
All generated queries must look through all subgraphs, by using "GRAPH ?g".

The Schema is in JSON format.
Include only necessary prefixes based on the below schema.
Never use any undefined prefixes.

Schema:
The OWL graph use the following prefixes:
{'owl:': 'http://www.w3.org/2002/07/owl#', 'rdf:': 'http://www.w3.org/1999/02/22-rdf-syntax-ns#', 'rdfs:':
'http://www.w3.org/2000/01/rdf-schema#', 'xsd:': 'http://www.w3.org/2001/XMLSchema#', 'eva:': 'http://www.
ericsson.com/eva#', 'skos:': 'http://www.w3.org/2004/02/skos/core#'}

The OWL graph supports the following node types:

{ "uri": "owl:AnnotationProperty", "comment": "None"},
{ "uri": "owl:Ontology", "comment": "None"},
{ "uri": "eva:bookmap", "comment": "None"},
{ "uri": "eva:document", "comment": "None"},
{ "uri": "eva:concept", "comment": "None"},
{ "uri": "eva:topic", "comment": "None"},
{ "uri": "eva:task", "comment": "None"},
{ "uri": "eva:fragment", "comment": "None"},
{ "uri": "eva:step", "comment": "None"},
{ "uri": "eva:product_name_product", "comment": "None"},
````
````

{ "uri": "owl:Class", "comment": "None"},
{ "uri": "eva:product_number_product", "comment": "None"},
{ "uri": "eva:product_number_rstate_product", "comment": "None"},
{ "uri": "owl:DatatypeProperty", "comment": "None"},
{ "uri": "rdf:Seq", "comment": "None"},
{ "uri": "owl:ObjectProperty", "comment": "None"},
{ "uri": "owl:TransitiveProperty", "comment": "None"},
{ "uri": "eva:image", "comment": "None"},
{ "uri": "eva:p3d", "comment": "None"},
{ "uri": "eva:tool", "comment": "None"},
{ "uri": "eva:video", "comment": "None"},
{ "uri": "owl:ontology", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#log", "comment": "None"},
{ "uri": "owl:Restriction", "comment": "None"}

The OWL graph supports the following object properties, i.e., relationships between objects:

{ "uri": "eva:contains", "comment": "None"},
{ "uri": "eva:hasColor", "comment": "None"},
{ "uri": "eva:hasFeaures", "comment": "None"},
{ "uri": "eva:hasIndicatorMode", "comment": "None"},
{ "uri": "eva:hasMarking", "comment": "None"},
{ "uri": "eva:hasOpticalIndicator", "comment": "None"},
{ "uri": "eva:hasPart", "comment": "None"},
{ "uri": "eva:hasPart_directly", "comment": "None"},
{ "uri": "eva:hasProductType", "comment": "None"},
{ "uri": "eva:hasSymbol", "comment": "None"},
{ "uri": "eva:has_alarm", "comment": "None"},
{ "uri": "eva:has_category", "comment": "None"},
{ "uri": "eva:has_commissioning_procedure", "comment": "None"},
{ "uri": "eva:has_component", "comment": "New Product built from to other Products"},
{ "uri": "eva:has_figure_id", "comment": "None"},
{ "uri": "eva:has_grounding_guideline", "comment": "None"},
{ "uri": "eva:has_image", "comment": "None"},
{ "uri": "eva:has_image_id", "comment": "None"},
{ "uri": "eva:has_managed_object", "comment": "None"},
{ "uri": "eva:has_mounting_alternative", "comment": "Information on mounting posssibility (Wall or Pole, etc.)"},
{ "uri": "eva:has_overview", "comment": "None"},
{ "uri": "eva:has_product", "comment": "None"},
{ "uri": "eva:has_product_type", "comment": "Product Type Classification"},
{ "uri": "eva:has_video", "comment": "None"},
{ "uri": "eva:hastoolList", "comment": "None"},
{ "uri": "eva:is_contained_by", "comment": "None"},
{ "uri": "eva:is_installed_as", "comment": "None"},
{ "uri": "eva:is_installed_in", "comment": "Environment (Indoor or Outdor)"},
{ "uri": "eva:is_raised_by", "comment": "None"},
{ "uri": "eva:item_number", "comment": "None"},
{ "uri": "eva:needTools", "comment": "None"},
{ "uri": "eva:partOf", "comment": "None"},
{ "uri": "eva:raises_alarm", "comment": "None"},
{ "uri": "eva:refers_to", "comment": "None"},
{ "uri": "eva:serves_rat", "comment": "Radio Access Technology (RAT)"},
{ "uri": "eva:toolImage", "comment": "None"},
{ "uri": "eva:toolName", "comment": "None"},
{ "uri": "eva:usesOpticalIndicator", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#includes_file", "comment": "None"}

The OWL graph supports the following data properties, i.e., relationships between objects and literals:

{ "uri": "eva:additional_text", "comment": "None"},
{ "uri": "eva:admonitions", "comment": "None"},
{ "uri": "eva:book_pdf", "comment": "None"},
{ "uri": "eva:bookdate", "comment": "None"},
{ "uri": "eva:bookmeta", "comment": "None"},
{ "uri": "eva:booknumber", "comment": "None"},
{ "uri": "eva:booktitle", "comment": "None"},
{ "uri": "eva:booktitlealt", "comment": "None"},
{ "uri": "eva:has_image", "comment": "None"},
{ "uri": "eva:has_product_number", "comment": "FAJ Number of Resource"},
{ "uri": "eva:index", "comment": "None"},

````
{ "uri": "eva:on_site_activity", "comment": "None"},
{ "uri": "eva:overview_context", "comment": "None"},
{ "uri": "eva:p3d_file", "comment": "None"},
{ "uri": "eva:p3d_tag", "comment": "None"},
{ "uri": "eva:p3d_url", "comment": "None"},
{ "uri": "eva:placement", "comment": "None"},
{ "uri": "eva:revision", "comment": "None"},
{ "uri": "eva:section", "comment": "None"},
{ "uri": "eva:task_context", "comment": "None"},
{ "uri": "eva:topic", "comment": "None"},
{ "uri": "eva:video_file", "comment": "None"},
{ "uri": "eva:video_tag", "comment": "None"},
{ "uri": "eva:video_url", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#file_sha-256", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#filename", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#source_version", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#time_elapsed", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#time_elapsed_(text)", "comment": "None"},
{ "uri": "http://www.ericsson.com/eva/log#timestamp", "comment": "None"}

The OWL graph supports the following annotation properties, i.e., comments or information about the nodes:

{ "uri": "http://swrl.stanford.edu/ontologies/3.3/swrla.owl#isRuleEnabled", "comment": "None"},
{ "uri": "eva:note", "comment": "None"},
{ "uri": "eva:text", "comment": "None"},
{ "uri": "eva:uri", "comment": "None"},
{ "uri": "skos:prefLabel", "comment": "None"}

Note: Be as concise as possible.
Do not include any explanations or assumptions or suggestions or notes or apologies in your responses.
Do not respond to any questions that ask for anything else than for you to construct a SPARQL query.
Do not include any text except the SPARQL query generated within backticks ``` and ```.
Do not include the backticks in the response. Only return the SPARQL query within backticks.

History:
[HumanMessage(content='How many ports are in Router 6471/2 DC?'), AIMessage(content='The Router 6471/2 DC does
not have any ports.'), HumanMessage(content='What is the weight of AIR 21 B4A B12P B5P?'), AIMessage(content="
I'm sorry, but I don't have any information regarding the weight of AIR 21 B4A B12P B5P."), HumanMessage
(content='What is its maximum frequency?'), AIMessage(content="I'm sorry, but I don't have any information
regarding the maximum frequency.")]

The question is:
What is the maximum frequency of AIR 21 B4A B12P B5P?


> Finished chain.
Generated SPARQL:
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX eva: <http://www.ericsson.com/eva#>

SELECT ?max_frequency
WHERE {
GRAPH <http://www.ericsson.com/eva/graph/hw-base>{
    ?prod rdfs:label ?product_label .
    FILTER(STR(?product_label) = "AIR 21 B4A B12P B5P")
    ?prod eva:has_max_frequency ?max_frequency .
    }
}
Full Context:
[]

> Finished chain.
-------------------------
Question: 'What is its maximum frequency?'
Last Answer: 'The maximum frequency of the AIR 21 B4A B12P B5P varies depending on the operating band. For Band
4, the maximum frequency is 2155 MHz. For Band 12, it is 746 MHz, and for Band 5, it is 894 MHz.'
Last tool used: 'vector_search_process'
Whether the question is answered (then add "answered" in UPPERCASE here), or you don't know the answer, or
there is no answer (then add "none" in UPPERCASE here): ANSWERED
-------------------------
````